

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0706

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards

(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 10

Total Marks:50

Annexure A

Coding challenge

Code of Conduct:

*I **B.Pardhu Arjun (192525245)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Arjun Basava

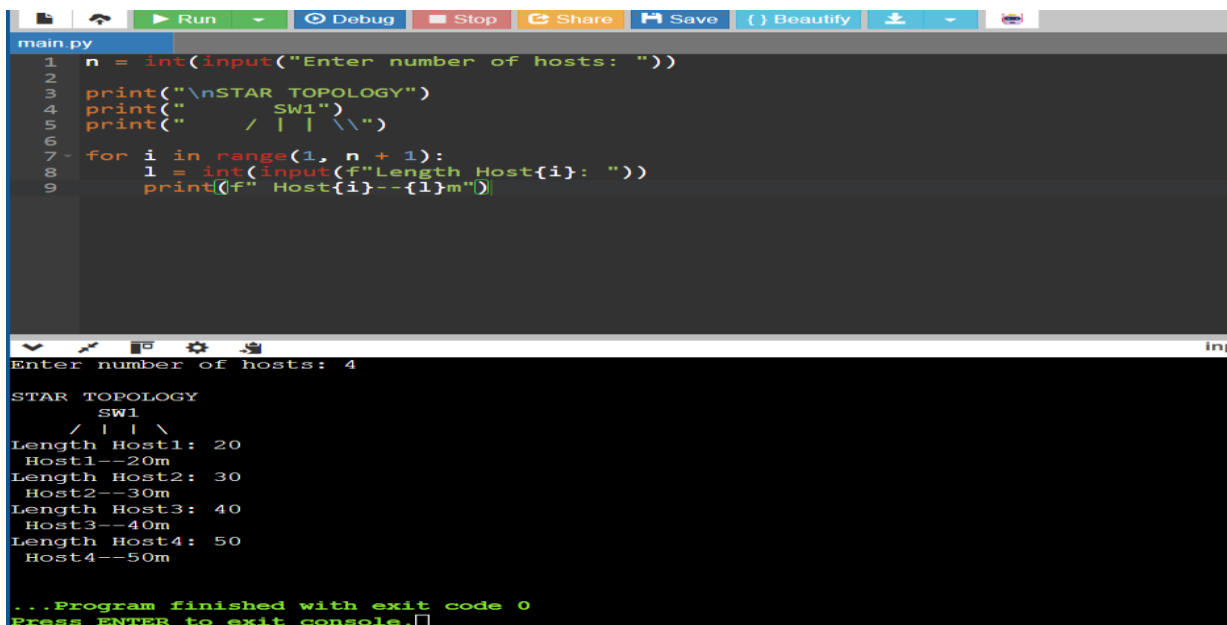
Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.
2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.
3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named `csma_log.txt`.
4. Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.
5. Create a class `StarSwitch` with methods `add_port(host_id)`, `remove_port(host_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

ANSWER:

1)



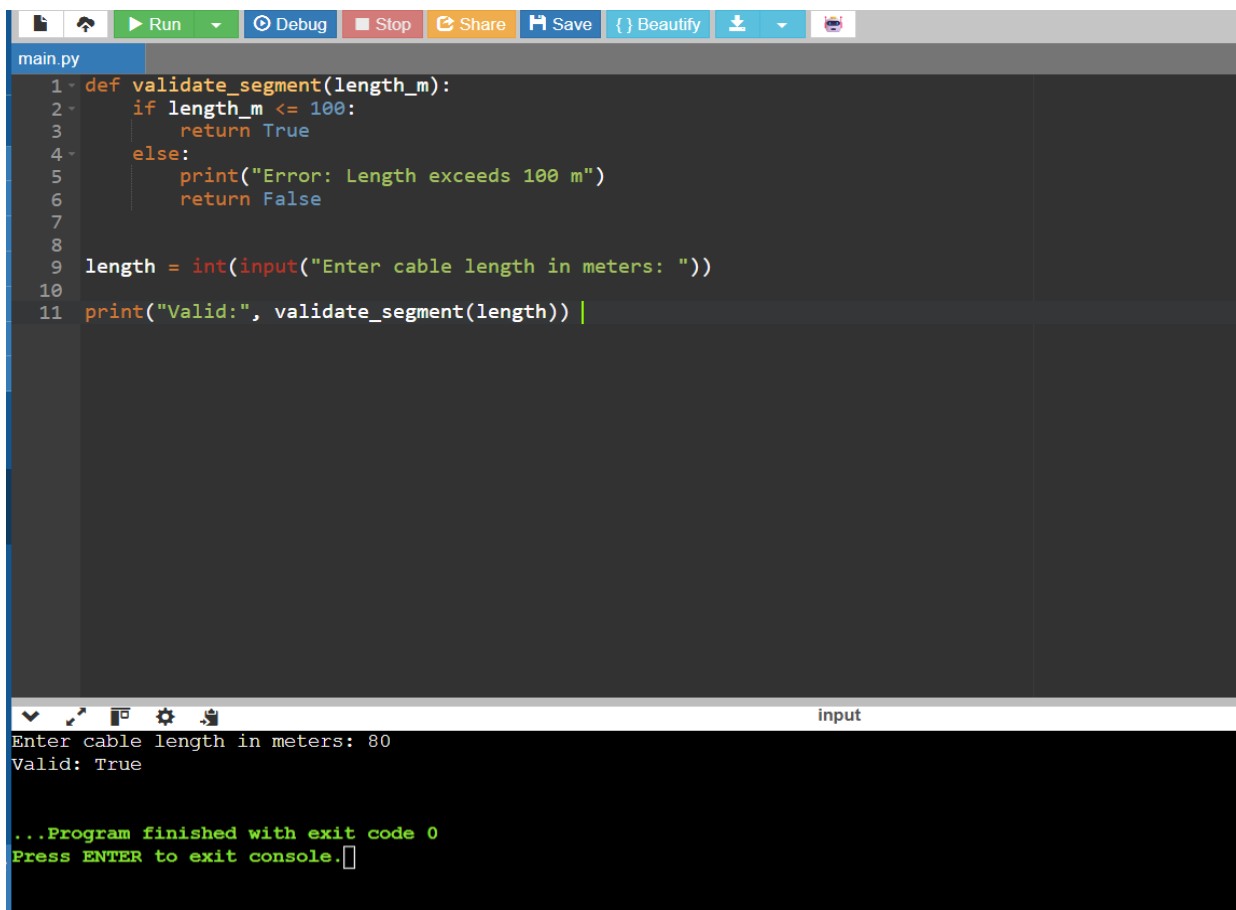
```
main.py
1 n = int(input("Enter number of hosts: "))
2
3 print("\nSTAR TOPOLOGY")
4 print("      SW1")
5 print("    /  |  \")
6
7 for i in range(1, n + 1):
8     l = int(input(f"Length Host{i}: "))
9     print(f"Host{i}--{l}m")

Enter number of hosts: 4

STAR TOPOLOGY
      SW1
    /  |  \
Length Host1: 20
Host1--20m
Length Host2: 30
Host2--30m
Length Host3: 40
Host3--40m
Length Host4: 50
Host4--50m

...Program finished with exit code 0
Press ENTER to exit console.
```

2)



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'validate_segment' that checks if a cable length is less than or equal to 100 meters. If it is, it returns True; otherwise, it prints an error message and returns False. The main part of the program prompts the user to enter a cable length in meters, converts it to an integer, and then prints the result of the validation function. The console output shows the user entering '80' and the program outputting 'Valid: True'. The program finishes with exit code 0.

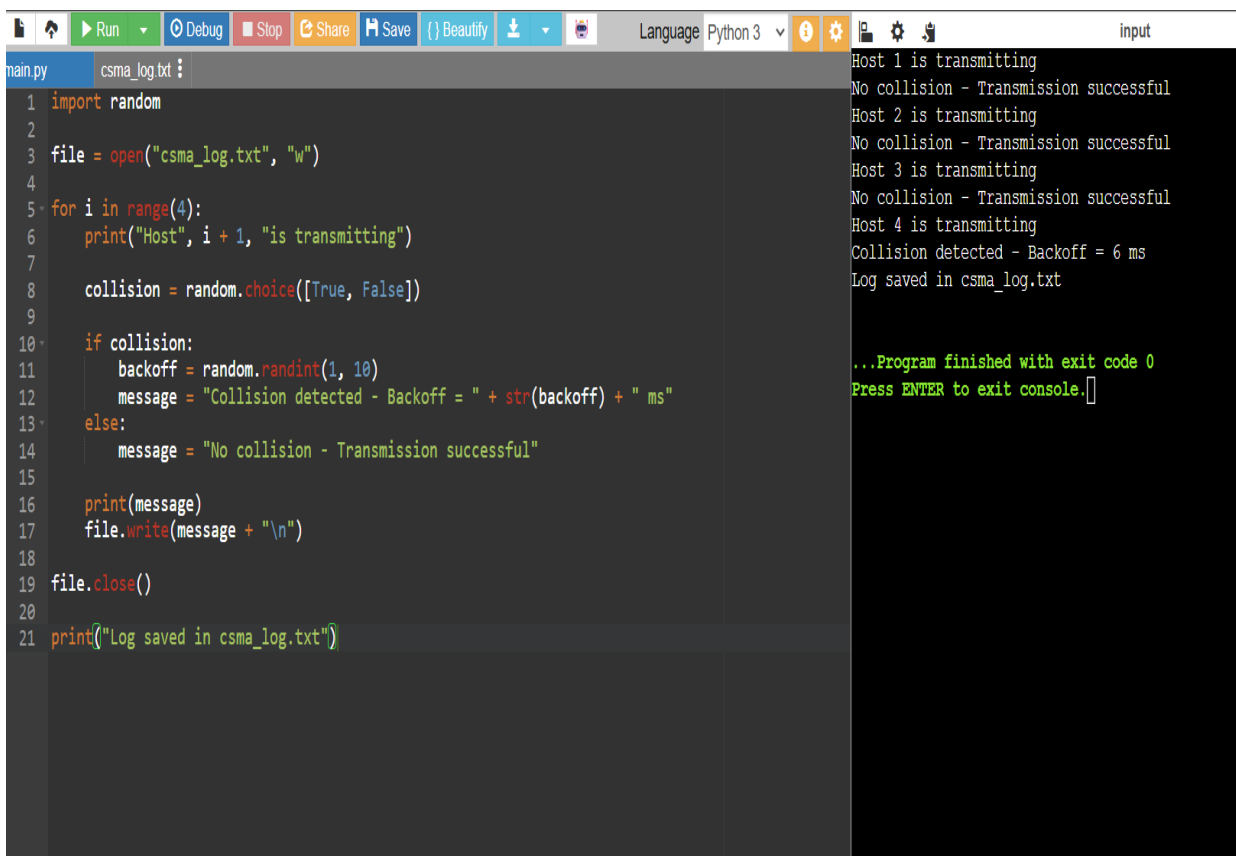
```
1 def validate_segment(length_m):
2     if length_m <= 100:
3         return True
4     else:
5         print("Error: Length exceeds 100 m")
6         return False
7
8
9 length = int(input("Enter cable length in meters: "))
10
11 print("Valid:", validate_segment(length))
```

input

Enter cable length in meters: 80
Valid: True

...Program finished with exit code 0
Press ENTER to exit console.

3)



The screenshot shows a Python IDE with a file named 'main.py'. The code simulates a CSMA/CD protocol. It imports the 'random' module and opens a file named 'csma_log.txt' for writing. It then enters a loop where it simulates four hosts. For each host, it prints a message indicating the host is transmitting. It then checks for a collision using 'random.choice([True, False])'. If a collision is detected, it generates a random backoff time between 1 and 10, prints a message indicating the collision and backoff, and writes this message to the log file. If no collision is detected, it prints a message indicating successful transmission and writes this message to the log file. After the loop, it closes the file and prints a message indicating the log has been saved. The console output shows the simulation results for four hosts, including a collision for the third host with a backoff of 6 ms. The program finishes with exit code 0.

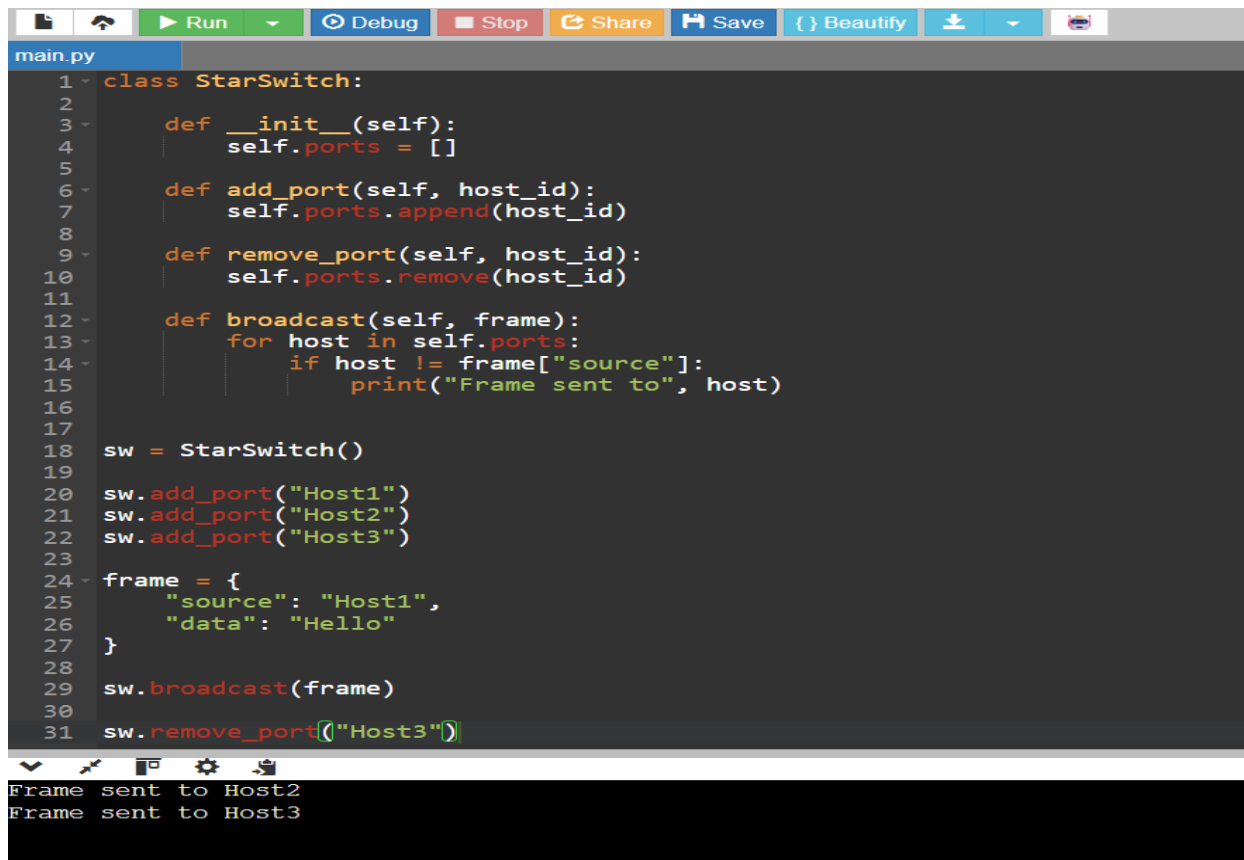
```
1 import random
2
3 file = open("csma_log.txt", "w")
4
5 for i in range(4):
6     print("Host", i + 1, "is transmitting")
7
8     collision = random.choice([True, False])
9
10    if collision:
11        backoff = random.randint(1, 10)
12        message = "Collision detected - Backoff = " + str(backoff) + " ms"
13    else:
14        message = "No collision - Transmission successful"
15
16    print(message)
17    file.write(message + "\n")
18
19 file.close()
20
21 print("Log saved in csma_log.txt")
```

input

Host 1 is transmitting
No collision - Transmission successful
Host 2 is transmitting
No collision - Transmission successful
Host 3 is transmitting
No collision - Transmission successful
Host 4 is transmitting
Collision detected - Backoff = 6 ms
Log saved in csma_log.txt

...Program finished with exit code 0
Press ENTER to exit console.

4)



The image shows a code editor window with a dark theme. The editor has a toolbar at the top with buttons for Run, Debug, Stop, Share, Save, and Beautify. The file name 'main.py' is visible in the top left corner. The code defines a class 'StarSwitch' with methods for adding and removing ports, and broadcasting frames. The script creates an instance of 'StarSwitch', adds three hosts ('Host1', 'Host2', 'Host3'), creates a frame with source 'Host1' and data 'Hello', broadcasts the frame, and then removes 'Host3'.

```
1 class StarSwitch:
2
3     def __init__(self):
4         self.ports = []
5
6     def add_port(self, host_id):
7         self.ports.append(host_id)
8
9     def remove_port(self, host_id):
10        self.ports.remove(host_id)
11
12    def broadcast(self, frame):
13        for host in self.ports:
14            if host != frame["source"]:
15                print("Frame sent to", host)
16
17
18 sw = StarSwitch()
19
20 sw.add_port("Host1")
21 sw.add_port("Host2")
22 sw.add_port("Host3")
23
24 frame = {
25     "source": "Host1",
26     "data": "Hello"
27 }
28
29 sw.broadcast(frame)
30
31 sw.remove_port("Host3")
```

Below the editor, the output of the program is displayed in a terminal window. It shows two lines of output: 'Frame sent to Host2' and 'Frame sent to Host3'.

```
Frame sent to Host2
Frame sent to Host3
```